Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No: 92301733012**

Aim: To understand the process of convolution over the image and apply over the classification problem

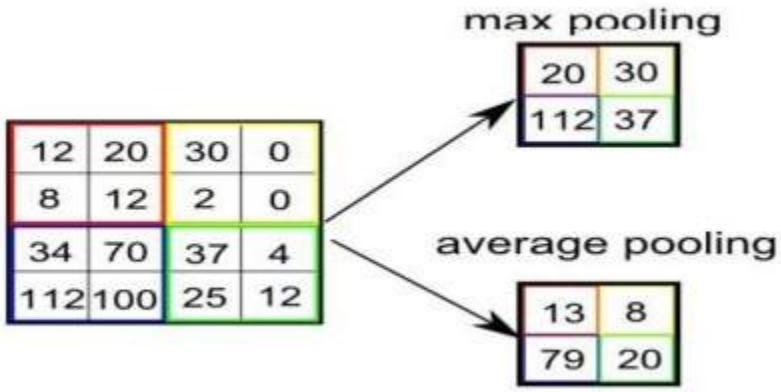
IDE: Google Colab

Theory:

Convolutional Neural Networks (CNN) are complex feed forward neural networks. CNNs are used for image classification and recognition because of its high accuracy There are three types of layers in a convolutional neural network: i. Convolutional layer ii. Pooling layer iii. Fully connected layer Each of these layers has different parameters that can be optimized and performs a different task on the input data.

What is Pooling Layer?

Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to decrease the computational power required to process the data through dimensionality reduction. There are two types of Pooling i. Average Pooling. ii. Max Pooling Max Pooling returns the maximum value from the portion of the image covered by the Kernel. On the other hand, Average Pooling returns the average of all the values from the portion of the image covered by the Kernel. Max Pooling also performs as a Noise Suppressant. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction. On the other hand, Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs a lot better than Average Pooling.



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No: 92301733012**

<u>What is Convolutional Layer?</u>

Convolutional layers are the major building blocks used in convolutional neural networks. A convolution is the simple application of a filter to an input that results in an activation. Repeated application of the same filter to an input results in a map of activations called a feature map, indicating the locations and strength of a detected feature in an input, such as an image. A convolutional layer contains a set of filters whose parameters need to be learned. The height and weight of the filters are smaller than those of the input volume. Each filter is convolved with the input volume to compute an activation map made of neurons.

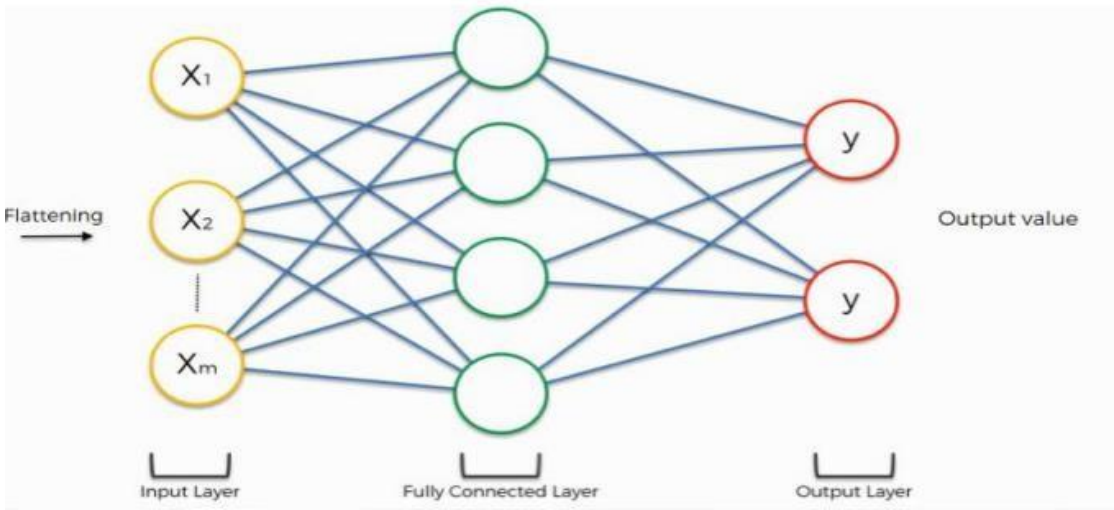
<u>What is Fully Connected Layer?</u>

A fully connected layer that takes the output of convolution/pooling and predicts the best label to describe the image We have three layers in the full connection step i. Input layer ii. Fully-connected layer iii. Output layer

Input Layer: It takes the output of the previous layers, “flattens” them and turns them into a single vector that can be an input for the next stage.

Fully Connected Layer: It takes the inputs from the feature analysis and applies weights to predict the correct label.

Output Layer: It gives the final probabilities for each label.



ReLU Layer: ReLU is an activation function. Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable. The main aim is to remove all the negative values from the convolution. All the positive values remain the same but all the negative values get changed to zero.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Normalize the data
5. Pre-process the data
6. Visualize the Data

7. Write the CNN model function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Observe the cost function vs iterations learning curve

Program (Code):

```
#import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import keras
from keras.datasets import fashion_mnist
import keras.models as models
import keras.layers as layers
from keras import regularizers
from keras.layers import Dropout

import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Dense,Activation,Flatten,Dropout,Conv2D, MaxPooling2D, LeakyReLU
from tensorflow.keras.utils import to_categorical   #for the categorical data
print(keras.__version__)

plt.imshow(train_X[0])
print(train_Y[0])

print(train_X.shape,train_Y.shape)
```

```
print(test_X.shape, test_Y.shape)

classes=np.unique(train_Y)
nClasses=len(classes)
print(classes)
print(nClasses)

train_X=train_X.reshape(-1,28,28,1)
test_X=test_X.reshape(-1,28,28,1)
print(train_X.shape,train_Y.shape)
print(test_X.shape, test_Y.shape)

#converting the values of the pixels from 0-1
#type-conversion
train_X=train_X.astype('float32')
test_X=test_X.astype('float32')
train_X=train_X/255
test_X=test_X/255

train_Y_one_hot=to_categorical(train_Y)
test_Y_one_hot=to_categorical(test_Y)
train_Y[0]
train_Y_one_hot[0]
from sklearn.model_selection import train_test_split
train_X,valid_X,train_label,valid_label=train_test_split(train_X,train_Y_one_hot,test_size=0.2,random_state=13)
train_X.shape,valid_X.shape,train_label.shape,valid_label.shape

batch_size=64
epochs=20
num_classes=10
```

```
model=tf.keras.Sequential()

#1st hidden layer
model.add(tf.keras.layers.Conv2D(32,(3,3),activation="linear",padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))
#2nd hidden layer
model.add(tf.keras.layers.Conv2D(64,(3,3),activation="linear",padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))
#3rd hidden layer
model.add(tf.keras.layers.Conv2D(128,(3,3),activation="linear",padding='same'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))

model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(128,activation='linear'))
model.add(tf.keras.layers.LeakyReLU(alpha=0.1))
model.add(tf.keras.layers.Dense(num_classes,activation='softmax'))#output final layer

model.compile(loss=tf.keras.losses.categorical_crossentropy,optimizer=tf.keras.optimizers.Adam(),metrics=['accur
acy'])
48000/64
model_train=model.fit(train_X,train_label,batch_size=64,epochs=20,verbose=1,validation_data=(valid_X,valid_la
bel))
testing_evaluation=model.evaluate(test_X,test_Y_one_hot)
testing_evaluation  #loss,accuracy

accuracy=model_train.history['accuracy']
val_accuracy=model_train.history['val_accuracy']
loss=model_train.history['loss']
```

```
val_loss=model_train.history['val_loss']
epochs=range(len(accuracy))

plt.plot(epochs,accuracy,label='training accuracy')
plt.plot(epochs,val_accuracy,label='validation accuracy')
plt.title('evaluating based on accuracy')
plt.legend()
plt.show()

plt.plot(epochs,loss,label='training loss')
plt.plot(epochs,val_loss,label='validation loss')
plt.title('evaluating based on loss')
plt.legend()
plt.show()

regmodel=tf.keras.Sequential()
#1st hidden layer
regmodel.add(tf.keras.layers.Conv2D(32,(3,3),activation="linear",padding='same'))
regmodel.add(tf.keras.layers.LeakyReLU(alpha=0.1))
regmodel.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))
regmodel.add(tf.keras.layers.Dropout(0.25))
#2nd hidden layer
regmodel.add(tf.keras.layers.Conv2D(64,(3,3),activation="linear",padding='same'))
regmodel.add(tf.keras.layers.LeakyReLU(alpha=0.1))
regmodel.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))
regmodel.add(tf.keras.layers.Dropout(0.25))
#3rd hidden layer
regmodel.add(tf.keras.layers.Conv2D(128,(3,3),activation="linear",padding='same'))
regmodel.add(tf.keras.layers.LeakyReLU(alpha=0.1))
regmodel.add(tf.keras.layers.MaxPooling2D(pool_size=(2,2),padding='same'))
regmodel.add(tf.keras.layers.Dropout(0.40))
```

```
regmodel.add(tf.keras.layers.Flatten())


regmodel.add(tf.keras.layers.Dense(128,activation='linear'))
regmodel.add(tf.keras.layers.LeakyReLU(alpha=0.1))
regmodel.add(tf.keras.layers.Dropout(0.3))
regmodel.add(tf.keras.layers.Dense(num_classes,activation='softmax'))#output final layer

regmodel.compile(loss=tf.keras.losses.categorical_crossentropy,optimizer=tf.keras.optimizers.Adam(),metrics=['accuracy'])
reg_model_train=regmodel.fit(train_X,train_label,batch_size=64,epochs=20,verbose=1,validation_data=(valid_X,valid_label))
```

Results:

To be attached with

a. Training dataset

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz

29515/29515 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz

26421880/26421880 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz

5148/5148 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz

4422102/4422102 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step

b. Model summary

```
regmodel.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(64, 28, 28, 32)	320
leaky_re_lu_4 (LeakyReLU)	(64, 28, 28, 32)	0
max_pooling2d_3 (MaxPooling2D)	(64, 14, 14, 32)	0
dropout (Dropout)	(64, 14, 14, 32)	0
conv2d_4 (Conv2D)	(64, 14, 14, 64)	18,496
leaky_re_lu_5 (LeakyReLU)	(64, 14, 14, 64)	0
max_pooling2d_4 (MaxPooling2D)	(64, 7, 7, 64)	0
dropout_1 (Dropout)	(64, 7, 7, 64)	0
conv2d_5 (Conv2D)	(64, 7, 7, 128)	73,856
leaky_re_lu_6 (LeakyReLU)	(64, 7, 7, 128)	0
max_pooling2d_5 (MaxPooling2D)	(64, 4, 4, 128)	0
dropout_2 (Dropout)	(64, 4, 4, 128)	0
flatten_1 (Flatten)	(64, 2048)	0
dense_2 (Dense)	(64, 128)	262,272
leaky_re_lu_7 (LeakyReLU)	(64, 128)	0
dropout_3 (Dropout)	(64, 128)	0
dense_3 (Dense)	(64, 10)	1,290

Total params: 1,068,704 (4.08 MB)
Trainable params: 356,234 (1.36 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 712,470 (2.72 MB)

c. Training and validation accuracy w.r.t epochs before regularization

d. Training and validation loss w.r.t epochs before regularization

e. Training and validation accuracy w.r.t epochs after regularization

Epoch 1/20

f. **750/750** ━━━━━━━━━━━━━━━━━━━━ **101s** 130ms/step - accuracy: 0.7784 - loss: 0.6000 - val_accuracy: 0.8613 - val_loss: 0.3794

g. Epoch 2/20

h. **750/750** ━━━━━━━━━━━━━━━━━━━━ **96s** 128ms/step - accuracy: 0.8615 - loss: 0.3820 - val_accuracy: 0.8815 - val_loss: 0.3201

i. Epoch 3/20

j. **750/750** ━━━━━━━━━━━━━━━━━━━━ **95s** 126ms/step - accuracy: 0.8765 - loss: 0.3326 - val_accuracy: 0.8932 - val_loss: 0.2854

k. Epoch 4/20

l. **750/750** ━━━━━━━━━━━━━━━━━━━━ **100s** 134ms/step - accuracy: 0.8871 - loss: 0.3056 - val_accuracy: 0.9037 - val_loss: 0.2622

m. Epoch 5/20

n. **750/750** ━━━━━━━━━━━━━━━━━━━━ **137s** 127ms/step - accuracy: 0.8944 - loss: 0.2834 - val_accuracy: 0.9077 - val_loss: 0.2511

o. Epoch 6/20

p. **750/750** ━━━━━━━━━━━━━━━━━━━━ **95s** 127ms/step - accuracy: 0.8961 - loss: 0.2760 - val_accuracy: 0.9107 - val_loss: 0.2457

q. Epoch 7/20

r. **750/750** ━━━━━━━━━━━━━━━━━━━━ **94s** 125ms/step - accuracy: 0.9004 - loss: 0.2628 - val_accuracy: 0.9135 - val_loss: 0.2403

s. Epoch 8/20

t. **750/750** ━━━━━━━━━━━━━━━━━━━━ **93s** 124ms/step - accuracy: 0.9062 - loss: 0.2497 - val_accuracy: 0.9081 - val_loss: 0.2442

u. Epoch 9/20

v. **750/750** ━━━━━━━━━━━━━━━━━━━━ **144s** 127ms/step - accuracy: 0.9086 - loss: 0.2438 - val_accuracy: 0.9129 - val_loss: 0.2325

w. Epoch 10/20

x. **750/750** ━━━━━━━━━━━━━━━━━━━━ **95s** 126ms/step - accuracy: 0.9107 - loss: 0.2373 - val_accuracy: 0.9183 - val_loss: 0.2279

y. Epoch 11/20

z. **750/750** ━━━━━━━━━━━━━━━━━━━━ **94s** 125ms/step - accuracy: 0.9132 - loss: 0.2299 - val_accuracy: 0.9188 - val_loss: 0.2277

aa. Epoch 12/20

bb. **750/750** ━━━━━━━━━━━━━━━━━━━━ **146s** 131ms/step - accuracy: 0.9162 - loss: 0.2231 - val_accuracy: 0.9189 - val_loss: 0.2210

cc. Epoch 13/20

dd. **750/750** ━━━━━━━━━━━━━━━━━━━━ **138s** 126ms/step - accuracy: 0.9173 - loss: 0.2197 - val_accuracy: 0.9222 - val_loss: 0.2152

ee. Epoch 14/20

ff. **750/750** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **141s** 125ms/step - accuracy: 0.9186 - loss: 0.2163 - val_accuracy: 0.9215 - val_loss: 0.2146

gg. Epoch 15/20

hh. **750/750** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **147s** 132ms/step - accuracy: 0.9204 - loss: 0.2134 - val_accuracy: 0.9203 - val_loss: 0.2169

ii. Epoch 16/20

jj. **750/750** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **137s** 126ms/step - accuracy: 0.9204 - loss: 0.2104 - val_accuracy: 0.9235 - val_loss: 0.2236

kk. Epoch 17/20

ll. **750/750** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **147s** 133ms/step - accuracy: 0.9229 - loss: 0.2048 - val_accuracy: 0.9268 - val_loss: 0.2052

mm. Epoch 18/20

nn. **750/750** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **98s** 131ms/step - accuracy: 0.9214 - loss: 0.2036 - val_accuracy: 0.9184 - val_loss: 0.2271

oo. Epoch 19/20

pp. **750/750** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **98s** 131ms/step - accuracy: 0.9239 - loss: 0.2016 - val_accuracy: 0.9227 - val_loss: 0.2179

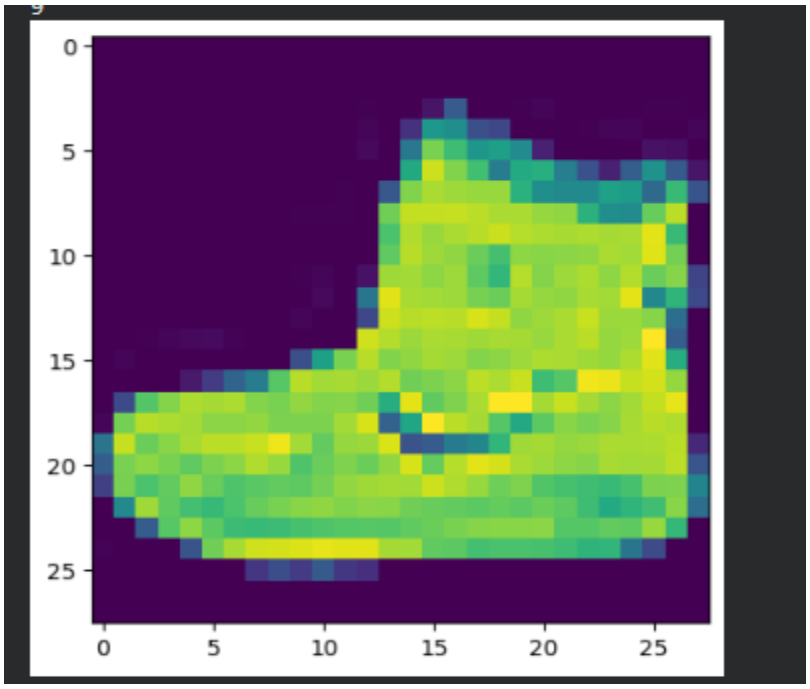
qq. Epoch 20/20

rr. **750/750** ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **97s** 130ms/step - accuracy: 0.9247 - loss: 0.2002 - val_accuracy: 0.9258 - val_loss: 0.2079

f. Training and validation loss w.r.t epochs after regularization

g. Original v/s predicted labels for correct predicted observations

h. Original v/s predicted labels for incorrect predicted observatio



-

Pre Lab Exercise:

a. What is Convolution process? Explain giving example.

Convolution is a mathematical operation used in image processing and CNNs to extract important features from input data. It works by sliding a small matrix called a filter (kernel) over the input image, performing element-wise multiplication, and summing the results to create a feature map. This process helps detect patterns such as edges, textures, and shapes while preserving spatial relationships in the image.

Example:

Suppose we have a small **3×3 image matrix**:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

And a **2×2 filter (kernel)**:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$(1 \times 1) + (2 \times 0) + (4 \times 0) + (5 \times 1) = 6$$

The result **6** becomes the first value in the output (feature map).

b. What are different layers in CNN model?

The layers in CNN are:

Convolutional Layer

- Applies filters (kernels) to the input image.
- Extracts features like edges, textures, and patterns.

Activation Layer (ReLU)

- Applies a non-linear function (usually ReLU).
- Helps the model learn complex patterns.

Pooling Layer (Subsampling Layer)

- Reduces the size of feature maps.
- Common types: Max Pooling and Average Pooling**.**
- Helps reduce computation and prevent overfitting.

Flatten Layer

- Converts 2D feature maps into a 1D vector.
- Prepares data for fully connected layers.

Fully Connected (Dense) Layer

- Performs classification based on extracted features.

Output Layer

- Produces the final prediction.
- Uses activation functions like **Softmax** (multi-class) or **Sigmoid** (binary classification).

c. What is the requirement of the pooling layer in the CNN model?

1. Dimensionality Reduction

   Reduces the width and height of feature maps, which lowers computation and memory usage.

2. Reduce Overfitting

   By reducing parameters, it helps prevent the model from memorizing the training data.

3. Feature Selection

   Keeps the most important information (e.g., Max Pooling keeps the strongest features).

4. Translation Invariance

   Makes the model less sensitive to small shifts or distortions in the image.

d. What is the requirement of the use of ReLU activation function after convolution step?

The ReLU (Rectified Linear Unit) activation function is applied after the convolution layer to introduce non linearity into the model.

Its main requirements are:

1. Introduce Non-Linearity

   Convolution is a linear operation. ReLU makes the network capable of learning complex, non-linear patterns.

2. Remove Negative Values

   ReLU converts all negative values to zero, keeping only positive activations.

3. Improve Training Speed

   ReLU is computationally simple and helps the model train faster.

4. Reduce Vanishing Gradient Problem

   Compared to sigmoid or tanh, ReLU helps maintain stronger gradients during backpropagation.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No: 92301733012**

Observation and Result Analysis:

e. Nature of the dataset

The dataset used is Fashion-MNIST, which consists of grayscale images of size 28×28 pixels. It contains 10 classes representing different types of clothing (e.g., shoes, shirts, bags, etc.).

The dataset is:

- ✓ Balanced (each class has similar number of samples)
- ✓ Structured and labeled (supervised learning)
- ✓ Image-based, requiring feature extraction via convolution
- ✓ Pixel values range from 0 to 255, and normalization to 0–1 improves training stability.
- ✓ The dataset is relatively simple but not trivial, making it suitable for testing CNN performance.

f. Training Process without regularization

During training without regularization, the model learns patterns from the training data and achieves high training accuracy. However, it may also start memorizing the data instead of generalizing, which leads to overfitting. As a result, the validation accuracy is lower and validation loss is higher compared to training performance.

g. After regularization in the training Process

After applying regularization techniques (like dropout or L2 regularization), the model becomes more generalized. The gap between training and validation accuracy reduces, and the model performs better on unseen data. Although training accuracy may slightly decrease, validation accuracy improves, indicating better generalization.

h. Observation over the Learning Curves

From the learning curves, it is observed that without regularization, there is a large gap between training and validation curves, indicating overfitting. After applying regularization, both curves become closer, showing improved model stability and performance. The loss decreases steadily, and accuracy increases more consistently.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No: 92301733012**

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No: 92301733012**

Post Lab Exercise:

a. Why CNN is preferred over ANN for images

Convolutional Neural Networks are preferred for image processing because they preserve the spatial structure of images, capturing patterns like edges, textures, and shapes. Unlike Artificial Neural Networks, they use local connectivity and weight sharing, which reduces complexity and the number of parameters. They also learn hierarchical features (from simple edges to complex objects) and provide translation invariance, meaning objects can be recognized regardless of position.

b. Can CNN be applied over Text data? If yes, then how. If no, then why?

Yes.

How it works:

- Text is first converted into numerical form using word embeddings.
- The sentence becomes a matrix (words × features).
- CNN applies 1D convolution filters over the sequence to capture patterns like phrases or n-grams.
- Pooling layers select the most important features.
- The output is passed to a classifier for tasks like sentiment analysis or text classification.

c. What is the role of dropout layer?

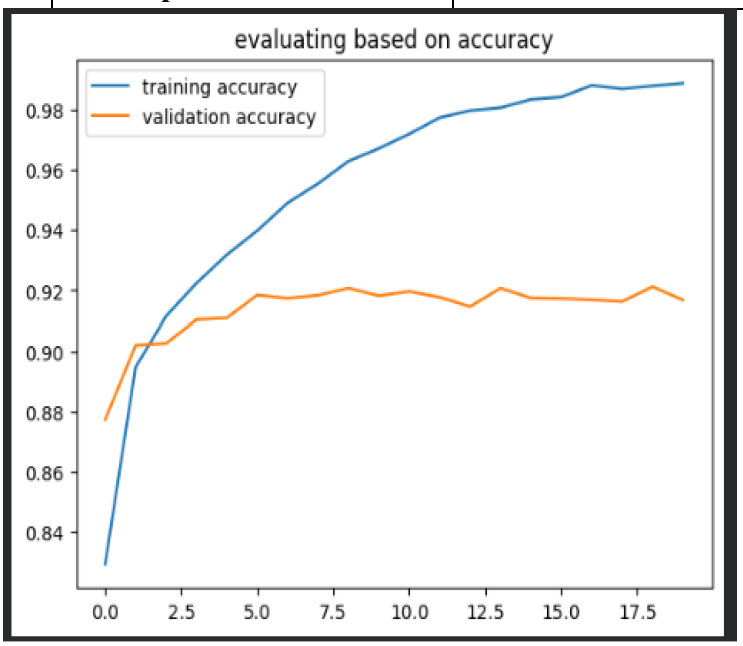
The dropout layer is a regularization technique that randomly disables some neurons during training, preventing the model from relying too heavily on specific nodes. This reduces overfitting, encourages the learning of robust and distributed features, and improves the model's ability to generalize to new, unseen data.

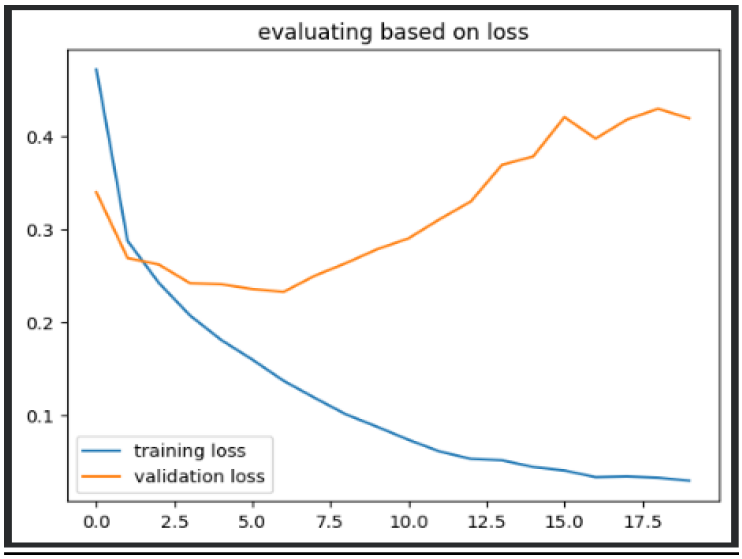
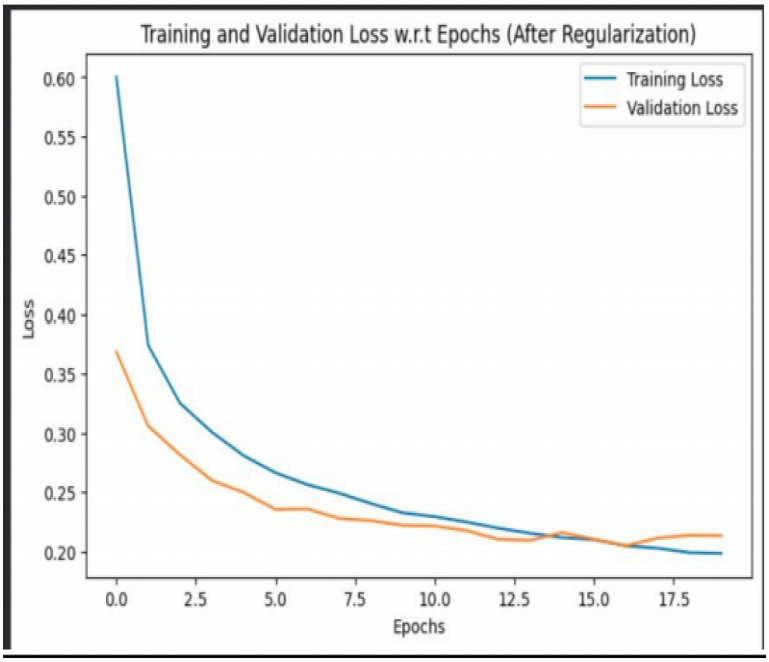
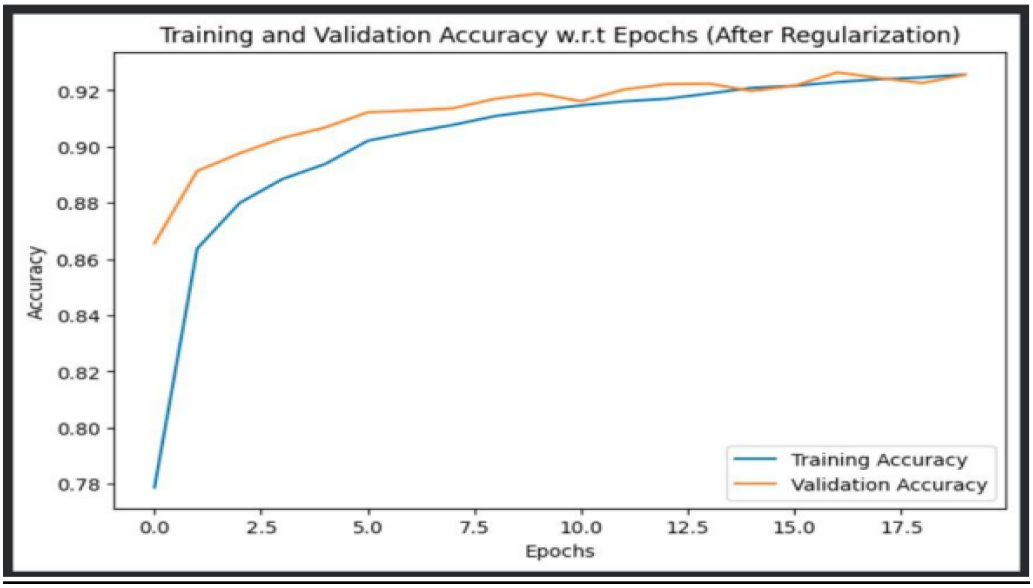
d. What will happen if maxpooling is replaced with minpooling?

If max pooling is replaced with min pooling in a Convolutional Neural Networks, the network will start selecting the smallest values from each region instead of the most significant ones. Since max pooling is designed to capture the strongest features such as edges and important patterns, using min pooling causes the model to focus on less important or background information. As a result, the quality of feature extraction decreases and the overall performance of the model is likely to be negatively affected.

Results:

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No: 92301733012**





Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No: 92301733012**